

FAISS-Based Retrieval Optimization for Low-Resource Arabic Question Answering

Akram Taha

[Affiliation]

[Department]

[City, Country]

akramtaha30@gmail.com

Abstract—Open-domain question answering (QA) systems for Arabic remain constrained by the scarcity of large labeled datasets and by the computational cost of dense retrieval over growing document collections. FAISS is the de facto library for approximate nearest-neighbor (ANN) search in dense retrieval pipelines, but its index configuration (Flat, IVF, HNSW) exposes a three-way tradeoff between retrieval accuracy, query latency, and memory footprint that is rarely characterized for morphologically rich, low-resource languages such as Arabic. This paper studies that tradeoff directly. We build a retrieval pipeline over a multilingual sentence-embedding model and evaluate three FAISS index families, plus a BM25 sparse baseline, under a common protocol measuring top- k retrieval accuracy, query latency, and on-disk index size on two Arabic QA corpora (ARCD and the Arabic portion of TyDi QA) and their pooled combination. Contrary to the common assumption that Arabic morphology makes lexical retrieval unreliable, we find that BM25 outperforms the multilingual dense retriever by 10–15 recall points at every k and on every corpus tested, while HNSW matches exact (Flat) search accuracy almost exactly and IVF-PQ trades a small, consistent accuracy loss for the smallest index footprint, an effect traced to under-trained product-quantization codebooks at low-resource corpus scale. All reported results were independently reproduced on a second, architecturally distinct hardware platform, yielding numerically identical retrieval metrics. We report the full comparison, the accuracy-latency curves behind it, and release the evaluation code, discussing the implications of these results for retrieval system design in resource-constrained Arabic QA deployment.

Index Terms—information retrieval, FAISS, approximate nearest neighbor search, Arabic natural language processing, question answering, low-resource languages, dense retrieval

I. INTRODUCTION

Retrieval-augmented question answering separates the task of finding relevant evidence from the task of extracting or generating an answer, and the quality of the first stage bounds the quality of everything downstream. In practice, a QA system cannot answer a question correctly from a passage it never retrieves, so the retriever functions as a hard ceiling on end-to-end accuracy [1]. For high-resource languages such as English, dense retrievers trained on large question-passage pairs and served over efficient ANN indexes have pushed that ceiling high enough that the bottleneck has largely shifted to the reader. Arabic QA has not reached the same point. Labeled Arabic QA data is an order of magnitude smaller than its English counterparts [2], Arabic morphology is commonly assumed to produce a large and sparse surface vocabulary

that degrades lexical retrieval methods such as BM25 [3] relative to dense alternatives, and dense retrievers for Arabic typically rely on multilingual embedding models rather than language-specific ones trained on Arabic-scale data. We test this assumption directly in §V and do not find it holds for the general-purpose multilingual encoder studied here.

A second, less discussed bottleneck sits below the embedding model: the ANN index used to serve retrieval at query time. FAISS [4] is the library most commonly used for this step, and it offers index families with fundamentally different tradeoffs. A flat (exhaustive) index gives exact nearest-neighbor results but scales linearly with corpus size, which is unacceptable once a document collection reaches even a few hundred thousand passages on modest hardware. Inverted-file (IVF) indexes, built on product quantization [5], partition the embedding space into coarse clusters and search only a subset of them, trading a controlled amount of accuracy for large speedups. Graph-based indexes such as HNSW [6] instead build a navigable small-world graph over the embeddings and typically offer a better accuracy-latency curve than IVF at the cost of higher memory use and slower index construction. These tradeoffs are well studied for large-scale English or general-purpose benchmarks, but they are rarely characterized for a QA-sized, low-resource Arabic corpus, where corpus size, index parameters, and available compute all look different from the billion-scale settings the FAISS literature typically targets.

This distinction matters operationally. Groups building Arabic QA systems for low-resource deployment settings, for example on limited server hardware, on-premise infrastructure without GPUs, or in contexts where document collections are modest (tens of thousands to low millions of passages) rather than web-scale, need a concrete answer to a practical question: which FAISS index configuration gives the best retrieval accuracy per unit of latency and memory for their setting, and how much accuracy is actually lost by choosing a fast approximate index over an exact one. Existing FAISS benchmarks answer this question for generic vector search or for high-resource NLP; this paper answers it specifically for Arabic QA retrieval.

We make the following contributions. First, we describe a retrieval pipeline for low-resource Arabic QA built on a multilingual sentence embedding model and FAISS, along

with the full construction procedure for three index configurations spanning the accuracy-latency-memory space (Flat, IVF, HNSW), plus a BM25 sparse baseline. Second, we specify and execute an experimental protocol evaluating these configurations along three axes simultaneously, top- k retrieval accuracy, query latency, and index size, rather than accuracy alone, which is the more common but incomplete practice in prior retrieval evaluations, on two Arabic QA corpora and their pooled combination. Third, we report the resulting comparison and two findings that run counter to the assumptions this paper started from: BM25 outperforms a general-purpose multilingual dense retriever by a wide margin on this task, and IVF-PQ’s usual accuracy-memory tradeoff is measurably worse at low-resource corpus scale due to undertrained product-quantization codebooks, while HNSW is effectively lossless relative to exact search. Fourth, we release the evaluation code and cached embeddings, and confirm reproducibility directly by re-executing the full pipeline on a second, architecturally distinct machine, obtaining metrics identical to those reported here. The remainder of the paper is organized as follows. Section II reviews related work in dense retrieval, FAISS-based ANN search, and Arabic NLP resources. Section III describes the embedding model, index configurations, and dataset. Section IV specifies the experimental protocol and metrics. Section V reports results. Section VI discusses deployment implications, and Section VII concludes with directions for future work.

II. RELATED WORK

A. Dense Retrieval

Dense passage retrieval replaces sparse lexical matching with learned embeddings and a dual-encoder architecture, encoding questions and passages into a shared vector space and retrieving by nearest-neighbor search. Karpukhin et al. [1] showed that a dual-encoder trained on question-passage pairs outperforms BM25 by a wide margin on English open-domain QA benchmarks, and dense retrieval has since become the default first stage in most open-domain QA and retrieval-augmented generation pipelines. A recurring assumption in this line of work is that the ANN search step itself is effectively free or exact, an assumption that holds for small evaluation corpora but breaks down once the retriever is deployed over a realistically sized document collection, which is exactly the regime this paper studies.

B. Approximate Nearest Neighbor Search and FAISS

FAISS [4] implements a family of ANN index structures with different accuracy-speed-memory tradeoffs, and its GPU and CPU implementations are widely adopted in both academic retrieval systems and production search infrastructure. Two index families are directly relevant to this work. IVF indexes rely on product quantization [5], which decomposes the embedding space into a Cartesian product of low-dimensional subspaces, quantizes each subspace independently, and represents each vector by a short code; combined with an inverted file that restricts search to a subset of coarse clusters

($nprobe$), IVF indexes trade recall for large reductions in search time and memory. HNSW [6] instead builds a multi-layer proximity graph over the dataset and performs greedy graph traversal at query time, which in practice yields a stronger recall-latency curve than IVF-PQ variants but at a higher memory cost per vector and a slower build phase, since the full graph structure and vector data are typically kept resident. Most published comparisons of these index families target billion-scale, general-purpose vector search (e.g., SIFT or GIST image descriptors, or synthetic benchmarks); this paper instead targets a QA-corpus-scale, Arabic-specific setting, where the operating point (smaller N , smaller compute budget, latency-sensitive query-time deployment) differs from the regimes typically reported in the FAISS benchmarking literature.

C. Arabic NLP Resources and Low-Resource Retrieval

Labeled Arabic QA resources are comparatively scarce. The Arabic Reading Comprehension Dataset (ARCD) [2] provides roughly 1,400 crowd-sourced question-answer pairs over Arabic Wikipedia articles and remains one of the primary extractive Arabic QA benchmarks. Multilingual QA benchmarks such as TyDi QA [7] and its cross-lingual extension XOR-QA [8] include Arabic among their typologically diverse language set and have been used to evaluate retrieval and answer extraction jointly across languages, though Arabic is one of several languages covered rather than the primary focus. Because Arabic-specific dense retrievers trained at the scale of DPR’s English training data do not yet exist, most Arabic retrieval pipelines instead rely on multilingual sentence embedding models such as multilingual MiniLM, trained via knowledge distillation from monolingual sentence encoders [9], or LaBSE [10], which is trained explicitly for language-agnostic bitext retrieval across more than 100 languages. Recent work has begun to study retrieval-augmented generation specifically in Arabic and cross-lingual Arabic-English settings, including analyses of retrieval bias and quality degradation when Arabic queries are matched against mixed-language corpora [11], [12]. This literature establishes that the embedding side of Arabic retrieval is an active and only partially solved problem; the ANN indexing side that this paper addresses has received comparatively little dedicated study in the Arabic setting.

III. METHODOLOGY

A. Embedding Model

We use paraphrase-multilingual-MiniLM-L12-v2, a 12-layer, 118M-parameter multilingual sentence embedding model trained via knowledge distillation from a monolingual teacher model onto a multilingual student using parallel data [9]. The model maps text to a 384-dimensional dense vector space and supports Arabic among its 50+ covered languages. We select this model over larger multilingual encoders such as LaBSE [10] because its smaller embedding dimensionality and parameter count are representative of what a low-resource deployment is realistically able to run without GPU inference, which is consistent with the

deployment-oriented framing of this paper. We report results for this single embedding configuration in this paper; a LaBSE ablation to test whether the index-level conclusions are sensitive to embedding choice is left to future work (§VII). All passages and questions are encoded with the same model and pooling strategy (mean pooling over token embeddings, L2-normalized) to keep the encoding side of the pipeline fixed while the FAISS index configuration is varied.

B. FAISS Index Configurations

We evaluate three FAISS [4] index configurations, chosen to span the accuracy-latency-memory tradeoff space rather than to exhaustively cover every configuration FAISS supports:

- **IndexFlatIP / IndexFlatL2 (Flat):** exhaustive exact search over all passage embeddings. This configuration serves as the accuracy upper bound (ground-truth nearest neighbors) against which the approximate indexes are measured, and as the latency and memory lower bound to beat.
- **IndexIVFPQ (IVF):** an inverted-file index with product quantization [5]. We vary the number of coarse centroids (`nlist`) and the number of clusters probed at query time (`nprobe`) to trace out an accuracy-latency curve rather than reporting a single operating point, since `nprobe` is the primary knob a deployment would tune.
- **IndexHNSWFlat (HNSW):** a hierarchical navigable small-world graph index [6] built directly over full-precision embeddings. We vary the graph connectivity parameter (`M`) and the search-time exploration parameter (`efSearch`) to trace a comparable accuracy-latency curve.

All three indexes are built from the same passage embedding matrix, so any observed differences in accuracy, latency, or memory are attributable to the indexing structure rather than to differences in the underlying embeddings.

C. Dataset

We use the Arabic Reading Comprehension Dataset (ARCD) [2] as the primary evaluation corpus, supplemented by the Arabic portion of TyDi QA’s gold-passage task [7] to test whether index-level conclusions transfer across Arabic QA datasets built with different annotation methodologies (crowd-sourced Wikipedia questions for ARCD versus information-seeking queries collected without translation for TyDi QA). Passages are segmented at the paragraph level, consistent with the unit of retrieval used in the original dataset construction, and questions are paired with their gold supporting passage to compute retrieval accuracy against a known-correct target. ARCD contributes 465 passages and 1,395 questions; the Arabic subset of the TyDi QA gold-passage validation split contributes 842 passages and 921 questions. We additionally pool both corpora (1,307 passages, 2,316 questions) into a combined condition that approximates a somewhat larger and more heterogeneous low-resource deployment corpus than either dataset alone provides; we report this alongside the two single-dataset conditions rather than in place of them; all

three are well below the “tens of thousands to low millions” deployment regime motivated in §I, and we return to this scale gap as a limitation in §VI. Text is normalized before encoding: Arabic diacritics (tashkil) and tatweel elongation characters are stripped, alef variants are unified to bare alef, and ta marbuta / alef maqsura are normalized to their unmarked counterparts.

IV. EXPERIMENTAL SETUP

A. Evaluation Metrics

We evaluate every index configuration along three axes simultaneously:

- **Retrieval accuracy:** top- k recall (the gold passage appears among the top k retrieved passages) for $k \in \{1, 5, 10, 20\}$, and mean reciprocal rank (MRR) of the gold passage.
- **Query latency:** mean and 95th-percentile wall-clock latency per query, measured over single-query (non-batched) search to reflect an interactive QA deployment setting, averaged over repeated runs to control for measurement noise.
- **Index size:** on-disk and in-memory size of the built index, reported separately from the size of the raw embedding matrix so that the overhead each index structure adds is visible.

Reporting accuracy alone, as much prior ANN benchmarking work does, would hide the latency and memory cost of achieving that accuracy; reporting latency alone would hide how much retrieval quality is sacrificed to achieve it. We report all three jointly and construct accuracy-latency curves for the tunable configurations (IVF, HNSW) rather than single-point comparisons, so that a practitioner can read off the operating point appropriate to their own latency or memory budget.

B. Hardware and Implementation

All indexes are built and queried using `faiss-cpu 1.14` to reflect a low-resource deployment setting without GPU acceleration, on a 4-core aarch64 CPU with 3.8GB RAM. Embedding generation uses `transformers` with a manual mean-pooling head reproducing the sentence-transformers pooling configuration for this model (PyTorch 2.9, CPU build). Index build time is recorded separately from query latency, since build time is a one-time offline cost while query latency is incurred on every request. All measurements, including latency, use 4 CPU threads throughout (`torch.set_num_threads(4)`); a single-threaded configuration was not additionally isolated, so the latency figures in §V should be read as a fixed-thread-count comparison across index types rather than an absolute single-core figure.

To verify that these results do not depend on the reference platform, the full pipeline (embedding, index construction, IVF-PQ/HNSW parameter sweeps, and the BM25 baseline) was independently re-executed end to end on a second machine with an unrelated CPU architecture (Apple Silicon, ARM64, single-threaded FAISS and PyTorch configuration to avoid a known OpenMP runtime conflict between the two libraries on that platform). Every recall@ k and MRR value produced by

this second run matched the corresponding value reported in Tables I and II exactly; absolute latencies differed, as expected given the different core counts and threading configurations between the two machines, but the relative ordering of index configurations was unchanged. This cross-platform check gives us confidence that the reported accuracy figures reflect the retrieval pipeline itself rather than an artifact of a specific runtime environment.

C. Experimental Protocol

For each corpus (ARCD, TyDi QA-Arabic, and their pooled Combined condition) and each index configuration, we: (1) encode all passages with the fixed embedding model and build the index; (2) encode all evaluation questions with the same model; (3) run top- k retrieval for every question and record whether the gold passage is recovered at each k ; (4) measure per-query latency over a warm cache, discarding the first five queries to exclude cold-start effects; (5) measure the final on-disk index size via a full index serialization. For IVF-PQ, $nlist$ is set to $\min(N/5, 32)$ for ARCD, 48 for TyDi QA-Arabic, and 64 for Combined (scaling with corpus size N), with $m = 48$ PQ subquantizers (8 bits each) fixed by the embedding dimensionality (384); $nprobe$ is swept over $\{1, 2, 4, 8, 16, 32\}$ (extended to 48/64 for the larger corpora). For HNSW, $M = 32$ and $efConstruction = 40$ are fixed and $efSearch$ is swept over $\{8, 16, 32, 64, 128\}$. We additionally run a BM25 [3] sparse-retrieval baseline over the same normalized passages and questions, using identical recall@ k /MRR/latency measurement code, to test the claim in §I that Arabic morphology degrades lexical retrieval relative to dense methods rather than leaving it untested. Evaluation code, cached embeddings, and the exact parameter grids are released alongside this paper (§VII).

V. RESULTS

Table I reports retrieval accuracy, mean query latency, and index size for all four retrieval methods (Flat, IVF-PQ at its best tested $nprobe$, HNSW at its best tested $efSearch$, and BM25) on all three corpora. The full accuracy-latency sweep behind the IVF-PQ and HNSW rows is given in Table II for the primary (ARCD) corpus.

TABLE I

RETRIEVAL ACCURACY, LATENCY, AND INDEX SIZE ON ARCD, TyDi QA-ARABIC, AND THEIR POOLED COMBINED CORPUS. IVF-PQ AND HNSW ROWS REPORT THEIR BEST TESTED OPERATING POINT ($nprobe=MAX$, $efSearch=128$); FULL SWEEPS IN TABLE II. BM25 HAS NO VECTOR INDEX, SO NO SIZE IS REPORTED.

Corpus	Method	R@1	R@5	R@10	R@20	MRR	Lat. (ms)	Size (KB)
ARCD (N=465)	Flat	30.6	51.0	58.3	65.1	.402	0.022	698
	IVF-PQ	28.5	51.3	57.8	64.5	.388	0.046	458
	HNSW	30.6	51.0	58.3	65.1	.402	0.048	821
	BM25	45.9	70.5	75.9	79.7	.569	0.219	–
TyDi-AR (N=842)	Flat	44.5	62.1	68.9	73.1	.522	0.036	1263
	IVF-PQ	41.5	60.7	66.9	72.5	.502	0.053	503
	HNSW	44.5	62.0	68.8	73.0	.521	0.060	1486
	BM25	54.8	72.2	78.1	82.0	.629	0.331	–
Combined (N=1307)	Flat	32.3	50.4	56.4	61.9	.404	0.051	1961
	IVF-PQ	30.1	48.5	54.8	59.7	.384	0.060	552
	HNSW	32.3	50.4	56.4	61.9	.404	0.067	2307
	BM25	44.0	66.5	72.4	77.3	.540	0.536	–

Three findings stand out. First, BM25 outperforms the multilingual dense retriever at every k and on every corpus, by 10–15 recall points at R@20 and by roughly 0.14–0.17 in MRR. This is the opposite of what the framing in §I assumed following the general (largely English-language) dense-retrieval literature [1]: a general-purpose multilingual sentence encoder, not fine-tuned for retrieval or for Arabic specifically, does not automatically beat a well-tuned lexical baseline on this task. Second, HNSW is effectively lossless relative to exact Flat search at $efSearch = 128$ on all three corpora (recall and MRR match Flat to within rounding), while IVF-PQ gives up a small but consistent amount of accuracy (0.6–1.6 R@20 points, 1.4–2.0 MRR points) even at its largest tested $nprobe$. Third, index size shows the expected ordering (IVF-PQ smallest, HNSW largest, Flat in between) and latency differences between the three FAISS configurations are all sub-tenth-of-a-millisecond at this corpus scale, meaning index choice has a far smaller practical effect here than the dense-vs-BM25 choice does.

TABLE II

ARCD ACCURACY-LATENCY SWEEP. IVF-PQ OVER $nprobe$ ($nlist=32$); HNSW OVER $efSearch$ ($M=32$).

IVF-PQ	R@20	MRR	Lat. (ms)
$nprobe=1$	33.3	.263	0.038
$nprobe=2$	43.5	.313	0.037
$nprobe=4$	54.3	.360	0.039
$nprobe=8$	60.0	.379	0.040
$nprobe=16$	64.0	.388	0.045
$nprobe=32$	64.5	.388	0.046
HNSW	R@20	MRR	Lat. (ms)
$efSearch=8$	63.7	.398	0.011
$efSearch=16$	64.6	.400	0.013
$efSearch=32$	64.8	.401	0.018
$efSearch=64$	65.0	.402	0.028
$efSearch=128$	65.1	.402	0.048

The sweep in Table II shows the two approximate index families behaving differently as their search-time budget grows. HNSW reaches within 1 R@20 point of its ceiling already at $efSearch = 8$, at roughly a quarter of the latency of its own $efSearch = 128$ setting, so most of the accuracy is available cheaply. IVF-PQ needs a much larger relative increase in $nprobe$ (probing nearly the whole index, since $nlist = 32$ here) to approach its own ceiling, and that ceiling remains below Flat and HNSW. We traced this to FAISS’s own training diagnostics: building the IVF-PQ codebooks on a corpus this size triggers FAISS’s internal warning that clustering 465–1307 points into 256 per-subquantizer centroids has far fewer training points than recommended ($\geq 9,984$ for 256 centroids per FAISS’s own guidance). Product quantization is

a data-hungry technique, and at the passage counts realistic for a single low-resource-language QA dataset, its codebooks are undertrained; this is a corpus-scale effect specific to small deployments rather than a general property of IVF-PQ, and it would be expected to close as corpus size grows toward the regime IVF-PQ is normally benchmarked at [5].

Evaluation code, cached embeddings, and full result JSON (including all sweep points omitted from Table II for space) are released alongside this paper.

VI. DISCUSSION

The central question this paper set out to answer was which FAISS configuration gives the best accuracy retained per unit of latency and memory spent, at the corpus scale and hardware budget realistic for low-resource Arabic QA deployment. The results in §V answer a related, and in our view more consequential, question first: at this scale, the choice of *retrieval method* (dense embedding vs. lexical BM25) dominates the choice of *index structure*. BM25 beats the multilingual dense retriever by a wide margin on every corpus and every k , while the gap between the best FAISS configuration (HNSW, effectively tied with exact Flat search) and the worst (IVF-PQ) is an order of magnitude smaller. A team building a low-resource Arabic QA system today, working from this result alone, would be better served asking “should we use BM25, a fine-tuned Arabic retriever, or hybrid retrieval” before asking “which FAISS index.” This ordering runs contrary to the motivation set out in §I, which followed the general dense-retrieval literature [1] in treating morphology-driven BM25 degradation as a known cost of sparse retrieval; our results do not support that assumption for this embedding model on this task. Two likely, non-exclusive explanations account for the gap: *paraphrase-multilingual-MiniLM-L12-v2* is a general-purpose paraphrase/semantic-similarity model, not a retriever trained on question-passage relevance pairs the way DPR-style encoders are [1]; and ARCD/TyDi-QA-Arabic questions frequently share exact surface-form overlap with their gold passage (named entities, dates, technical terms), a regime where BM25’s exact lexical matching has a structural advantage that a semantic encoder does not automatically inherit. Distinguishing between these explanations, and testing whether a retrieval-tuned or Arabic-specific dense encoder closes the gap, is direct future work (§VII).

Within the FAISS comparison itself, HNSW matches exact search accuracy almost exactly while IVF-PQ gives up a small, consistent amount of recall even at its largest tested n_{probe} . We traced this to undertrained PQ codebooks: FAISS’s own training diagnostics flag that clustering a few hundred to 1,300 points into 256 centroids per subquantizer has an order of magnitude fewer training points than recommended. This is a genuine, actionable finding for low-resource deployment specifically: IVF-PQ’s compression advantage (roughly 35–75% smaller than Flat across our three corpora) comes at a larger accuracy cost than the FAISS literature’s typical billion-scale benchmarks would suggest, precisely because those benchmarks train PQ codebooks on far more data than a

single low-resource-language corpus provides. HNSW requires no such data-hungry training step, an advantage that only became apparent in light of these results, and this may make it the safer default for small corpora even though it is the largest index of the three we evaluate.

Absolute latencies across all three FAISS configurations are sub-tenth-of-a-millisecond at our corpus scale (465–1,307 passages), so the accuracy-latency tradeoff we designed the study around is real but numerically small here; it would be expected to widen as corpus size grows toward the “tens of thousands to low millions” regime motivated in §I, which our corpora do not reach (§III-C). The ranking of index configurations (HNSW $\approx$ Flat $>$ IVF-PQ on accuracy) is stable across ARCD, TyDi QA-Arabic, and their pooled Combined corpus despite the three differing in passage-length distribution and annotation methodology, which supports treating this ranking as a general recommendation for low-resource Arabic QA deployment rather than an artifact of one dataset’s characteristics. The dense-vs-BM25 gap is similarly stable across all three corpora; of the two stable findings, this is the one with the larger practical consequence for retrieval system design.

VII. CONCLUSION AND FUTURE WORK

This paper evaluated FAISS index configurations (Flat, IVF-PQ, HNSW) and a BM25 baseline for low-resource Arabic question answering, under a three-axis protocol (accuracy, latency, index size) applied to two Arabic QA corpora and their pooled combination. Two findings emerged that were not anticipated at the outset of the study. First, a general-purpose multilingual dense retriever underperformed BM25 by 10–15 recall points at every k on every corpus tested, contrary to the framing in §I that followed the general dense-retrieval literature in treating BM25 as the weaker option for Arabic; for practitioners deploying Arabic QA under resource constraints, this suggests BM25, hybrid sparse-dense retrieval, or a retrieval-tuned Arabic embedding model deserve serious consideration before defaulting to a general-purpose multilingual encoder. Second, within FAISS itself, HNSW matched exact search accuracy almost exactly while IVF-PQ paid a small but consistent accuracy cost traceable to undertrained product-quantization codebooks at low-resource corpus scale, a scale-dependent effect distinct from IVF-PQ’s usual billion-scale characterization. The ranking of index configurations was stable across both source corpora and their combination, supporting these as general recommendations for low-resource Arabic QA deployment rather than dataset-specific artifacts.

Future work includes: testing whether a retrieval-tuned encoder (e.g., a DPR-style dual encoder trained on Arabic question-passage pairs) or an Arabic-specific model closes the gap to BM25 that a general-purpose multilingual encoder does not; running the LaBSE ablation noted in §III-A to test sensitivity to embedding choice; extending the corpus scale toward the “tens of thousands to low millions” regime motivated in §I, which our present corpora (465–1,307 passages) do not reach, to test whether the IVF-PQ codebook-training

effect closes as intended; evaluating hybrid sparse-dense retrieval directly, given that BM25 and dense retrieval may make complementary errors; and extending the comparison to additional low-resource languages beyond Arabic. Evaluation code and cached embeddings are released alongside this paper to support this follow-on work directly.

Data and Code Availability

Evaluation code (data loading, Arabic normalization, embedding, index construction, and evaluation scripts) and the full result JSON (including all sweep points) are included in this paper’s submission package and are additionally made publicly available at github.com/Akramtaha98/faiss-arabic-qa.

The consolidated reproduction script was independently re-run end to end on a second, architecturally distinct machine (§IV-B); all reported recall@ k and MRR values were reproduced exactly. ARCD [2] and the TyDi QA gold-passage task [7] are publicly available from their original sources.

REFERENCES

- [1] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.
- [2] H. Mozannar, K. El Hajal, E. Maamary, and H. Hajj, “Neural Arabic question answering,” in *Proceedings of the Fourth Arabic Natural Language Processing Workshop (WANLP)*, 2019, pp. 108–118.
- [3] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [4] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [5] H. Jégou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.
- [6] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.
- [7] J. H. Clark, E. Choi, M. Collins, D. Garrette, T. Kwiatkowski, V. Nikolaev, and J. Palomaki, “TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 454–470, 2020.
- [8] A. Asai, J. Kasai, J. H. Clark, K. Lee, E. Choi, and H. Hajishirzi, “XOR QA: Cross-lingual open-retrieval question answering,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2021, pp. 547–564.
- [9] N. Reimers and I. Gurevych, “Making monolingual sentence embeddings multilingual using knowledge distillation,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [10] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, and W. Wang, “Language-agnostic BERT sentence embedding,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 878–891.
- [11] S. R. El-Beltagy and M. A. Abdallah, “Exploring retrieval augmented generation in Arabic,” arXiv:2408.07425, 2024.
- [12] C. Amiraz, Y. Fyodorov, E. Haramaty, Z. Karnin, and L. Lewin-Eytan, “The cross-lingual cost: Retrieval biases in RAG over Arabic-English corpora,” arXiv:2507.07543, 2025, accepted to ArabicNLP 2025.